

● تقرير تدقيق تقني شامل

تقرير تدقيق تطبيق VIXOR

مراجعة تفصيلية للمشاكل التقنية والأمنية والمعمارية مع خطة عمل عملية متكاملة
لإصلاح جميع الثغرات وتقوية البنية التحتية

6

منخفض

6

متوسط

8

عالٍ

5

حرج

الملخص التنفيذي

يقدم هذا التقرير الشامل نتائج تدقيق تقني مفصّل لتطبيق VIXOR، لوحة تحكم تداول عملات مشفرة مبنية بتقنيات React و TanStack Start ومتكاملة مع Binance و TwelveData و OpenRouter AI و Supabase. يغطي التقرير ثلاثة محاور: التحقق من حالة الإصلاحات السابقة، مسح شامل لأكثر من 50 ملف عبر 13 فئة مشكلة، وتحليل أمني ومعماري معمّق مع خطة عمل عملية متكاملة. أظهرت النتائج أن الإصلاحات الثلاثة السابقة استمرت بنجاح في الملفات المستهدفة. ومع ذلك، كشف المسح عن 25 مشكلة نشطة عبر 8 من أصل 13 فئة. الجدير بالملاحظة أن نظام المصادقة مُنفذ بشكل صحيح عبر requireSupabaseAuth الذي يتحقق من صحة JWT عبر Supabase، وأن جميع دوال الخادم في جميع النطاقات تستخدم هذا المiddleware. كما أن نظام تحديد المعدل متقدم ويستخدم Redis مع احتياطي في الذاكرة.

25

مشكلة نشطة مكتشفة

3 / 3

إصلاحات سابقة مؤكدة

170

حالة as any في 46 ملف

13 / 8

فئات تحتوي مشاكل

التحقق من الإصلاحات السابقة

تم إجراء ثلاث عمليات إصلاح سابقة عبر ثلاثة commits منفصلة. النتيجة: جميع الإصلاحات الثلاثة لا تزال سارية ولم تفقد بسبب أي عمليات لاحقة.

الإصلاح 1: إضافة LINK/USDT إلى سجل الأصول

تم الإصلاح

AssetRegistry - إضافة زوج LINK/USDT كامل

```
src/shared/asset-registry/types.ts:348-364
```

تم التحقق من وجود تعريف LINK/USDT كاملاً مع جميع الرموز الصحيحة: LINKUSDT للبينانس، و LINK/USDT ل TwelveData، و TradingView ل BINANCE:LINKUSDT. الزوج محدد ك popular: true مما يعني ظهوره في القوائم السريعة. كما تم التحقق من وجود طريقة find() آمنة بجانب get() التي تُلقِي خطأ.

الإصلاح 2: مقاومة خطاف الأسعار الحية للأزواج غير المعروفة

تم الإصلاح

useLivePrices - استخدام find() بدلاً من get()

src/shared/market-data/use-live-prices.ts:77,156,193

جميع الاستدعاءات تستخدم `AssetRegistry.find()` الذي يُعيد undefined بدلاً من إلقاء خطأ. السطر 77 يتخطى الأزواج غير المعروفة بأمان، والسطر 156 لنفس الغرض أثناء جلب أسعار REST، والسطر 193 داخل دالة `getPrice`. هذا يمنع انهيار التطبيق عند طلب أي زوج غير مسجل.

الإصلاح 3: إصلاح نموذج OpenRouter + آلية الاحتياطي المحلي

تم الإصلاح

runAnalysis - نموذج صالح + احتياطي محلي ثلاثي الطبقات

src/domains/analysis/server/run-analysis.ts:226,307-348

السطر 226 يستخدم القيمة الافتراضية `google/gemma-4-31b-it:free` بدلاً من النموذج غير الصالح. بلوك `try-catch` شامل (الأسطر 224-319) يلتقط أخطاء `OpenRouter` وينتقل إلى `runLocalAnalysisFallback`. الدالة الاحتياطية نفسها تحتوي على طبقة حماية إضافية: إذا فشل المحرك المحلي، يتم استخدام `generateFallbackResult` كملاذ أخير. كذلك تم التحقق من أن عدم وجود `OPENROUTER_API_KEY` ينتقل مباشرة للاحتياطي المحلي (السطر 181-189).

ملاحظة عن الإصلاح الثالث

الإصلاح الثالث كان جزءاً من `commit` كبير شمل أكثر من 150 ملف بسبب عمليات تنسيق `linting` آلية. لكن التغيير الوظيفي الفعلي اقتصر على ملف `run-analysis.ts` فقط. جميع الملفات الأخرى في ذلك `Commit` تم تعديلها فقط لأسباب تنسيقية (مسافات، فواصل منقوطة) وليس تغييرات وظيفية.

القسم الثالث

تقييم الأمان والمصادقة

تم فحص البنية الأمنية بالكامل. النتيجة الأساسية هي أن نظام المصادقة قوي ومُنَفَّذ بشكل صحيح، لكن هناك ثغرة محددة في وظيفة واحدة تسمح بحذف عناصر لا يملكها المستخدم.

نقاط قوة أمنية مؤكدة

`requireSupabaseAuth` (src/shared/supabase/auth-middleware.ts) يتحقق فعلياً من صحة JWT عبر `supabase.auth.getUser(token)` وليس فقط من صيغة الهيدر. جميع دوال الخادم في جميع النطاقات (moxi, trading, watchlist,) تستخدم هذا المiddleware. لا توجد مفاتيح API مكشوفة في الكود من جهة العميل. لا يوجد استخدام لـ `dangerouslySetInnerHTML` في أي ملف مسار.

ثغرة: removeFromWatchlist تحذف بدون التحقق من الملكية

S1

في السطر 119 من `watchlist/functions.ts`، دالة الحذف تستخدم `eq("id", data.itemId)` فقط دون التحقق من أن العنصر ينتمي لقائمة مراقبة المستخدم الحالي. هذا يعني أن مستخدماً يعرف UUID عنصر في قائمة مراقبة مستخدم آخر يمكنه حذفه. يجب إضافة التحقق عبر JOIN مع جدول `watchlists` والتأكد من `user_id`.

src/domains/watchlist/functions.ts:119

عالي

reorderWatchlist يستخدم supabaseAdmin الذي يتجاوز RLS

S2

في السطر 158، يتم استيراد `supabaseAdmin` (خدمة `admin` بدون قيود RLS) لتحديث ترتيب العناصر. رغم أن العناصر يتم التعامل معها بمعرفاتها فقط، فإن عدم التحقق من أن هذه العناصر تنتمي للمستخدم الحالي يشكل خطراً أمنياً. يجب التبديل لاستخدام `context.supabase` مع التحقق من الملكية.

src/domains/watchlist/functions.ts:158

متوسط

استخراج رمز المصادقة يدوياً من ملفات تعريف الارتباط في MOXI

S3

في مكون MOXI (الأسطر 431-449)، يتم تحليل document.cookie يدوياً للبحث عن رموز Supabase. هذه الطريقة هشة وقد تتعطل إذا تغيرت أسماء ملفات تعريف الارتباط. يُنصح باستخدام واجهة Supabase الرسمية لإدارة الجلسات.

-copilot-component.tsx:431-449

متوسط

تقييم نظام تحديد المعدل (Rate Limiting)

عكس الادعاءات، نظام تحديد المعدل في VIXOR متقدم ومتعدد الطبقات. يتضمن: SlidingWindowLimiter للتحقق لكل مستخدم (مُستخدم في MOXI و Copilot)، و RateLimiter بفواصل زمنية دنيا لكل مزود بيانات خارجي (Finnhub, TwelveData, Binance)، و RedisRateLimiter مدعوم بـ Upstash Redis مع احتياطي InMemory للبيانات التي لا تتوفر فيها Redis. هذا النظام يُعد كافياً لمعظم حالات الاستخدام الحالية.

القسم الرابع

تدقيق فئات المشاكل الثلاث عشرة

تم مسح أكثر من 50 ملف لتحديد المشاكل المنتمية لكل فئة. تم استخدام تصنيف رباعي: حرج يؤثر على الوظائف الأساسية، وعالي يؤثر بشكل كبير على تجربة المستخدم، ومتوسط يمثل مشاكل بصرية أو ثانوية، ومنخفض يمثل تحسينات مقترحة.

الفئات 1: عدم تناسق الشريط العلوي

1 الصفحة الرئيسية تستخدم تخطيطاً مخصصاً بدلاً من PageLayout الموحد

1

الصفحة الرئيسية تستخدم عنصر div مخصص بهيئة flex خاصة بدلاً من المكون المشترك PageLayout. هذا يُنتج تناقضاً بصرياً حيث تفتقر لعنصر الشريط العلوي المحاط بإطار والنظام المتسامل. الحل: توحيد الصفحة مع PageLayout.

index.tsx:590-620

متوسط

الفئتان 2 و 3: ودجات ثابتة وبيانات وهمية

2 صفحة التبديل (Swap) تعمل بالكامل ببيانات وهمية ثابتة

2

كائن PRICES ثابت بأسعار مجمدة (SOL: 145.23). المحفظة وهمية بعنوان ثابت ورصيد مزيف SOL 42.5. الأرصدة في MOCK_BALANCES ملفقة. سجل getSwapHistory() يحتوي 4 صفقات وهمية. حتى زر "تنفيذ التبديل" يستخدم Math.random() لمحاكاة النجاح والفشل. لا يوجد أي تكامل خلفي حقيقي في هذه الصفحة.

-swap-component.tsx:12-500

حرج

3 أقسام الفوركس في صفحة تفاصيل التوكن تعرض بيانات ثابتة

3

مكون ForexSections يعرض 3 أحداث اقتصادية ثابتة لا تتغير أبداً. بيانات قوة العملات تحتوي على 8 عملات بقيم ثابتة (USD: 72, EUR: 65). لا يتم جلب أي بيانات من API حقيقي.

-token-symbol-component.tsx:2094-2131

عالي

4 مقاييس التغير في 1 ساعة و 7 أيام تعرض دائماً شرطة "-"

4

الأسطر 1225 و 1235 تعرض MetricPill بقيمة ثابتة "-". رغم توفر هذه البيانات في Binance API. هذا يمنع تحليل الأداء قصير ومتوسط المدى.

-token-symbol-component.tsx:1225,1235

منخفض

الفئات 4 و 5 و 6 و 7: نظيفة تماماً

صفحة الرسوم البيانية تعمل مع CandlestickChart للكربتو و TradingViewChart للفوركس. واجهة نتائج التحليل احترافية مع رسم FVG و Order Block على Canvas. النتائج تأتي من محرك AI حقيقي (OpenRouter أو المحرك المحلي) وليست مُعدة مسبقاً. صفحة إنشاء التحليل تستخدم createAnalysis من الخادم.

الفئة 8: نهج MOXI الخاطئ

5 MOXI مصمم كعقل AI لكنه يعمل كشات بسيط

MOXI واحد من 6 وكلاء متساويين يتم اختياره من قائمة منسدلة. رغم ادعاء قدرات "كشف استباقي" و"تنفيذ أدوات"، التنفيذ الفعلي واجهة محادثة قياسية فقط. لا يوجد سلوك مستقل أو مراقبة خلفية أو إطار عمل للأدوات. يجب تحويله لعقل AI نشط دائماً يقدم رؤى استباقية.

-copilot-component.tsx:55-165

حرج

6 لا توجد قدرات مستقلة أو إطار عمل للأدوات في MOXI

لا يوجد وظائف خلفية أو مهام مجدولة أو تنبيهات استباقية. المحادثة محدودة بـ 10 رسائل كحد أقصى (السطر 568). ملف agent-orchestrator.ts موجود في الخادم لكنه غير متصل بواجهة MOXI.

-copilot-component.tsx:160-162,568

عالٍ

الفئتان 9 و 10: الاكتشاف والفوركس

7 حجم تداول الفوركس دائماً صفر في صفحة الاكتشاف

volume24h: 0 دائماً مع تعليق "always 0 for forex". أزواج الفوركس ستظهر دائماً في أسفل القوائم عند الترتيب حسب الحجم.

-discover-forex-data.ts:137

منخفض

8 الفوركس لا يعرض شريط أسعار حية في صفحة الرسوم البيانية

useLivePrices يشترك فقط في أزواج الكربتو. عند اختيار فوركس، getPrice() يُرجع undefined وشريط السعر فارغ.

charts.tsx:59-63

متوسط

الفئتان 11 و 13: صفحات التوكنز

9 صفحات التوكنز المجهولة تظهر شبه فارغة

عند عدم وجود بيانات من API الاكتشاف، تعرض الصفحة رسم بياني فقط وزر تحليل بدون أي إحصائيات سوقية.

-token-symbol-component.tsx:486-708

متوسط

الفئة 12: بيانات الحائزين والمعاملات

بيانات الحائزين والمعاملات على السلسلة مُلَفقة بالكامل

10

عدد الحائزين بمعادلة وهمية: $0.001 + 500 \cdot \text{marketCap}$. تقسيم DEX/CEX ثابت 38%/62%. معاملات البيتان $\text{volume} =$ 5000000. الواجهة تعترف بكلمة "(simulated)" في السطر 2079. لا يوجد تكامل مع أي مزود بيانات على السلسلة حقيقي.

-token-symbol-component.tsx:1918-1921,2079

حرج

لا يوجد سجل معاملات حقيقي من البلوكتشين

11

صفحة التفاصيل تعرض فقط صفقات المستخدم الخاص. لا يوجد موجز معاملات أو قائمة تحويلات أو سجل صفقات DEX حقيقي.

-token-symbol-component.tsx

عال

القسم الخامس

مصفوفة ملخص الفئات الـ 13

#	الفئة	العدد	الخطورة	الملفات
1	عدم تناسق الشريط العلوي	1	متوسط	index.tsx
2	ودجات ثابتة	3	عال	-token-symbol-component.tsx, -swap-component.tsx
3	بيانات وهمية	7	حرج	-swap-component.tsx, -token-symbol-component.tsx
4	لا توجد رسوم بيانية	0	نظيف	جميع صفحات الرسوم تعمل
5	واجهة تحليل غير احترافية	0	نظيف	واجهة احترافية مع Canvas
6	نتائج تحليل خاطئة	0	نظيف	محرك AI حقيقي
7	مخرجات تحليل وهمية	0	نظيف	AI حقيقي من الخادم
8	نهج MOXI الخاطئ	2	حرج	-copilot-component.tsx
9	صفحة الاكتشاف	1	منخفض	-discover-forex-data.ts
10	انهيار الفوركس	2	متوسط	charts.tsx
11	صفحات التفاصيل معطلة	1	متوسط	-token-symbol-component.tsx
12	بيانات الحائزين مفقودة	4	حرج	-token-symbol-component.tsx
13	صفحات توكنز فارغة	1	متوسط	-token-symbol-component.tsx

القسم السادس

تحليل المشاكل المعمارية

الجذر: نمط MVP تحوّل لإنتاج بدون تنظيف

الأنماط المكتشفة تشير بوضوح إلى أن التطبيق بُني كنموذج أولي سريع ثم نُشر في الإنتاج دون تنظيف. صفحة السواب بالكامل وهمية (أسعار ثابتة، محفظة مزيفة، تنفيذ محاكى) وهو نمط كلاسيكي لبروتوتيب يُعرض للمستثمرين. المشكلة أن هذه الواجهة التجريبية بقيت في الإنتاج بدون توصيلها بالباك إند الحقيقي. رغم توفر المكتبات اللازمة (Solana Wallet Adapter مثبت بالفعل)، لم يتم التكامل.

A1 ملف `shared/data/index.ts` ضخّم بـ 1463 سطر ويجمع كل شيء

هذا الملف يحتوي على عشرات دوال الخادم لنطاقات مختلفة (Portfolio, Watchlist, Signals, Discovery, Whale, Alpha, PnL) وغيرها). هذا يجعل المشاكل تضيق فيه ويصعب الصيانة. يجب تقسيمه لملفات منفصلة لكل نطاق.

`src/shared/data/index.ts` (سطر 1463)

حرج

A2 صفحة التبديل معزولة تماماً عن البنية الخلفية

لا تستخدم أي من البنية التحتية الحقيقية (server functions, useQuery, Supabase, WebSocket). إعادة بنائها تتطلب تكاملاً مع Jupiter أو Redux Swap SDK ومحفظة Solana حقيقية عبر Wallet Adapter المثبت.

`-swap-component.tsx`

عالٍ

A3 عدم وجود خدمة مركزية لبيانات التوكنز

كل صفحة تجلب بياناتها بطريقة الخاصة أو تصنع بيانات وهمية. الحل: إنشاء TokenDataService مركزي يجمع من Birdeye و Helius و DexScreener ويوفرها عبر React Query.

البنية العامة

عالٍ

A4 170 حالة "as any" في 46 ملف تضعف Type Safety

التركيز الأعلى في `analysis-id-component.tsx` (20 حالة)، و `discover.tsx` (8) و `index.tsx` (9) و `premium.tsx` (5). هذه الحالات تتجاوز فحوصات TypeScript وتخفي أخطاء محتملة. يجب استبدالها بأنواع صحيحة أو `schemas Zod`.

`src/` ملف عبر 46

عالٍ

A5 MOXI يفتقر لبنية الوكيل/التنسيق رغم وجود البنية

`agent-orchestrator.ts` موجود في الخادم لكنه غير متصل بواجهة MOXI. يحتاج إطار عمل Tool Use ومهام خلفية للمراقبة الاستباقية ونظام تنبيهات يعرض الرؤى تلقائياً.

`-copilot-component.tsx + agent-orchestrator.ts`

عالٍ

نقاط معمارية إيجابية

معظم الصفحات (signals, pnl, portfolio, whale, alpha, perpetuals, curves, trade-desk) تستخدم `server functions + useQuery` بشكل صحيح. تكامل WebSocket عبر `BinanceWS` ممتاز. `Error Boundaries` موجودة. بيانات `vixor-mock.ts` تم تنظيفها. وحدة بيانات الفوركس تستخدم `Promise.allSettled` لمقاومة الأخطاء. نظام `Rate Limiting` متعدد الطبقات (`Redis + InMemory + Sliding Window`).

المكتبات والتبعيات الرئيسية

الفئة	المكتبة	الإصدار	الوظيفة في المشروع
الإطار	react	19.2.0	مكتبة واجهة المستخدم
	@tanstack/react-start	1.168.25	إطار التطبيق الكامل (SSR + routing)
	@tanstack/react-query	5.83.0	إدارة حالة الخادم والتخزين المؤقت
	tailwindcss	4.2.1	إطار التصميم والأنماط
	ai (Vercel AI SDK)	6.0.224	واجهة أنماط AI و generateObject
الذكاء الاصطناعي	@ai-sdk/openai-compatible	2.0.48	التوافق مع OpenRouter API
	zod	4.4.3	التحقق من صحة البيانات والأنماط
	@solana/web3.js	1.98.4	التفاعل مع شبكة Solana
	@solana/wallet-adapter-react	0.15.39	إدارة محفظة Solana (مثبت غير مُستخدم)
	viem / wagmi	2.53 / 3.6	دعم EVM chains
الرسوم البيانية	lightweight-charts	5.2.0	مكتبة TradingView للشموع اليابانية
	recharts	2.15.4	رسوم بيانية إضافية
	ccxt	4.5.64	واجهة موحدة للتبادلات (...Binance, OKX)
الأسواق	decimal.js	10.6.0	حسابات عشرية دقيقة للأسعار
	@supabase/supabase-js	2.107.0	المصادقة وقاعدة البيانات و RLS
	jsonwebtoken	9.0.3	إدارة رموز JWT
Telegram	@telegram-apps/sdk	3.11.8	تكامُل تطبيق Telegram Web App
المراقبة	@sentry/react	10.63.0	تتبع الأخطاء والأداء
	mixpanel-browser	2.80.0	تحليلات الاستخدام والسلوك
	@upstash/redis	1.38.0	تخزين مؤقت سحابي + Rate Limiting
التخزين	zustand	5.0.14	إدارة الحالة المحلية الخفيفة

خطة العمل المتكاملة (4 أنظمة)

تمثل الخطة التالية منهجية شاملة لإصلاح جميع المشاكل ومنع تكرارها. مقسمة لـ 4 أنظمة: الأمان الفوري، والصدق مع المستخدم، والبنية النظيف، وطبقة الاختبار التلقائي. يمكن استخدامها مباشرة كنقاط عمل لفريق التطوير أو لأداة AI مثل KIMI K3.

النظام 1: أمان فوري (أسبوع واحد)

إصلاح ثغرة حذف عناصر Watchlist الخاصة بمستخدمين آخرين

P0

في removeFromWatchlist أضف JOIN مع جدول watchlists وتأكد من user_id قبل الحذف. في reorderWatchlist استبدل supabaseAdmin بـ context.supabase وأضف التحقق من الملكية. الملف: src/domains/watchlist/functions.ts:119,158

استبدال تحليل document.cookie اليدوي بواجهة Supabase الرسمية

P1

في مكون MOXI استبدل التحليل اليدوي لملفات تعريف الارتباط باستخدام getSession() أو supabase.auth.getUser() من مكتبة Supabase الرسمية. الملف: copilot-component.tsx:431-449

إضافة التحقق من المدخلات في صفحة التبديل

P1

التحقق من أن المبلغ لا يتجاوز الرصيد، وأنه أكبر من 0، وأن الزوج المحدد صالح، والانزلاق السعري ضمن حدود معقولة. الملف: swap-component.tsx:~432

النظام 2: الصديق مع المستخدم (حرج للثقة)

إعادة بناء صفحة التبديل بالكامل أو وضع علامة Demo واضحة

P0

خياران: (أ) تكامل حقيقي مع Jupiter DEX + محفظة Solana حقيقية عبر @solana/wallet-adapter-react (ب) وضع علامة "وضع تجريبي" واضحة على كل عنصر وتعطيل زر التنفيذ. مفيش حل نص نص - ده موضوع ثقة المستخدم.

استبدال البيانات المُلققة ببيانات على السلسلة حقيقية

P0

إزالة المعادلات الوهمية (الحائزون، DEX/CEX، الحيتان) واستبدالها بـ API حقيقي: Helius RPC أو Birdeye API للحائزين والمعاملات، DexScreener API لتقسيم الحجم. الملف: token-symbol-component.tsx:1915-2080

استبدال بيانات الفوركس الثابتة ببيانات حية

P1

إزالة التقويم الاقتصادي الثابت وقوة العملات واستبدالها بـ API حقيقي (TwelveData أو ForexFactory). الملف: token-symbol-component.tsx:2090-2131

النظام 3: بنية نظيفة تمنع تكرار المشكلة

تقسيم shared/data/index.ts (1463 سطر) لملفات منفصلة لكل نطاق

P0

فكك الملف الضخم لـ: data/pnl.ts, data/alpha.ts, data/whale.ts, data/signals.ts, data/watchlist.ts, data/portfolio.ts. كل ملف يستورد requireSupabaseAuth بشكل مستقل. هذا يسهل المراجعة ويمنع اختفاء المشاكل في ملف ضخم.

نقل كل Mock Data لمجلد /src/mocks مع Feature Flag

P1

أنشئ مجلد /src/mocks وانقل كل بيانات PRICES و MOCK_BALANCES و useMockWallet إليه. أضف متغير بيئة VIXOR_DEMO_MODE واستخدمه شرطياً في الكود. أضف قاعدة ESLint تمنع استيراد mocks في كود الإنتاج.

إعادة تصميم MOXI كعقل AI مستقل

P1

ربط واجهة MOXI بـ agent-orchestrator.ts الموجود مسبقاً. إضافة إطار Tool Use، ومراقبة خلفية للأسواق عبر WebSocket، وتنبيهات استباقية تعرض الرؤى تلقائياً. الملفات: copilot-component.tsx + server/agent-orchestrator.ts

إنشاء خدمة مركزية TokenDataService

P1

خدمة موحدة تجمع من Birdeye و Helius و DexScreener وتوفرها لجميع الصفحات عبر React Query. يمنع تكرار الكود ويضمن تناسق البيانات.

تقليل حالات "as any" من 170 إلى أقل من 20

P1

البدء بالملفات الأكثر تضرراً: (5) premium.tsx، (8) discover.tsx، (9) index.tsx، (20) analysis-id-component.tsx. استبدال بأنواع TypeScript صحيحة أو Zod schemas.

إضافة اختبارات أمان تمنع رجوع الثغرات

P1

اختبار 1: تأكيد أن requireAuth يرفض tokens غير صالحة. اختبار 2: تأكيد أن getWatchlistData لا يُرجع بيانات مستخدمين آخرين. اختبار 3: تأكيد أن removeFromWatchlist لا يستطيع حذف عنصر مستخدم آخر. إضافتها لـ CI بجانب eslint و tsc.

إضافة قاعدة ESLint تمنع استيراد mocks في الإنتاج

P1

قاعدة no-restricted-imports تمنع استيراد أي شيء من /src/mocks في ملفات الإنتاج. لو حد استورد mock في component حقيقي، الـ lint هيفشل فوراً.

تفعيل Branch Protection + PR Template

P2

إغلاق الدمج المباشر على main. إضافة قالب PR يتضمن: لا mock من خارج /src/mocks. كل server function جديدة فيها auth، لا as any من غير سبب موثق.

مهام تحسينية (P2-P3)

توحيد تخطيط الصفحة الرئيسية مع PageLayout

P2

استبدال الشريط العلوي المخصص في index.tsx بـ PageLayout الموحد. الملف: index.tsx:590-620 | الفئة: #1

إضافة أسعار حية للفوركس في صفحة الرسوم البيانية

P2

توسيع useLivePrices ليشارك في أزواج الفوركس عبر استطلاع REST من TwelveData. الملف: charts.tsx:59-63

تحسين صفحات التوكنز الفارغة

P2

عند عدم وجود بيانات، عرض رسالة واضحة مع اقتراحات بديلة بدلاً من صفحة شبه فارغة. الملف: token-symbol-component.tsx:486-708

جلب بيانات التغير في 1 ساعة و 7 أيام

P3

إضافة البيانات من Binance API بدلاً من "-" دائماً. الملف: token-symbol-component.tsx:1225,1235

استبدال قوائم الأزواج الثابتة بقوائم ديناميكية

P3

استبدال CRYPTO_PAIRS و FOREX_PAIRS الثابتة بـ AssetRegistry.popular() و byCategory("forex"). الملف: charts.tsx:14-38

معالجة حجم تداول الفوركس الصفري

P3

إيجاد مصدر حجم تقريبي أو إزالة عمود الحجم من ترتيب الفوركس. الملف: discover-forex-data.ts:137